

The loop in three moves

Browser Use: How LLM Agents Drive the Web

Week 1, Lecture 2 · Summer 2026

What we'll cover

- The perceive-decide-act loop: one full cycle
- Perceive: what the agent reads from the browser
- Decide: how the LLM picks an action
- Act: what runs, what comes back
- max_steps and the stopping condition
- Reading a run's history

The perceive-decide-act loop

One cycle, three moves

Every step of a browser-use run follows the same sequence:

1. **Perceive:** read the current browser state (URL, title, page content)
2. **Decide:** ask the LLM: given this state, what should I do next?
3. **Act:** execute the chosen action in the browser; collect the result

The result of the Act phase feeds the next Perceive phase.

The loop runs until the task is done or `max_steps` is reached.

Perceive

What the agent reads

At the start of each step, a `BrowserSession` captures a **browser-state summary**:

- **Current URL:** where the browser is
- **Page title:** the `<title>` element
- **Page content:** a serialized representation of the page (more in Week 2)
- **Optional screenshot:** if vision is enabled

The state is a snapshot, not a live feed. The agent reads it once per step, then acts.

The **memory** field carries notes forward; `evaluation_previous_goal` records whether the last step succeeded.

(browser-use contributors, 2026)

Decide

The LLM picks an action

The `MessageManager` assembles:

- The **system prompt**: the task, available actions, constraints
- The **browser-state summary**: what the page looks like right now
- **Recent history**: what happened in previous steps

The LLM must reply in a fixed shape called `AgentOutput`.

Why a fixed shape matters

- Free-text replies cannot be parsed reliably
- `AgentOutput` (a Pydantic model) has specific fields the framework can always read:
 - `evaluation_previous_goal`: did the last step work?
 - `memory`: notes to carry forward
 - `next_goal`: what this step aims to do
 - `action`: the action (or actions) to execute

Structure is what makes the loop reliable.

Act

The action runs in the browser

- The `action` field names one or more built-in actions: click, type, navigate, scroll, extract, done
- Each action has parameters (e.g., which element to click, what text to type)
- The action runs in the real browser via Playwright
- The result comes back as an `ActionResult`

The result (success, extracted content, or an error) becomes part of the next step's Perceive input.

The done action ends the loop

- When the agent is satisfied, it emits `done` as its action
- `done` carries the final answer as a string
- `history.final_result()` returns this string after the run
- If the agent never emits `done`, the loop ends at `max_steps`

max_steps and stopping

Controlling the loop

```
agent = Agent(  
    task="...",  
    llm=ChatOpenAI(model="gpt-4.1-mini"),  
    max_steps=10,    # stop after 10 steps even if not done  
)  
history = await agent.run()  
print(history.is_done())    # True if agent emitted done  
print(history.final_result()) # The answer, or None if not done
```

- Default `max_steps` is set in the agent configuration
- Hit the limit: the run ends; `is_done()` returns `False`
- The history still records everything that happened

Reading the history

```
for i, step in enumerate(history.history):  
    state = step.state  
    print(f"Step {i+1}: {state.next_goal}")  
    print(f"    memory: {state.memory}")
```

Each entry in `history.history` is one full perceive-decide-act cycle.

`step.state.next_goal` is what the LLM decided to do.

`step.result` is what came back from the browser.



"Browser Use enables AI agents to interact with the web the way humans do: visually, contextually, and reliably."

What's next

- **Section (this week):** Run three tasks, annotate each step as perceive-decide-act, read the history
- **Week 2:** How the agent sees a page: the DOM, the accessibility tree, and the numbered element index
- **Week 3:** BrowserSession in depth: what goes into the browser-state summary

Take-aways

- Each step of the loop is three moves: perceive the page, decide on an action, act in the browser
- The LLM never touches the browser directly; it only replies in a structured shape
- `AgentOutput` fields (evaluation, memory, next_goal, action) are how the decision is expressed
- `max_steps` is the safety valve; `done` is the clean exit
- The history records every step; it is your primary debugging tool